Memoria de Prácticas

Introducción al Aprendizaje Automático

Jesús Fernández López
Rafael Carlos Schoettel Vilchez

Abril, 2026

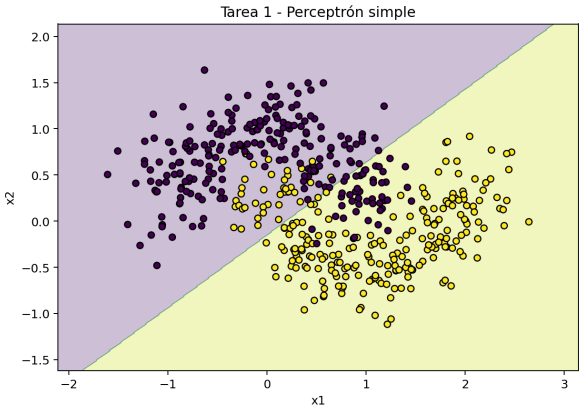
Arquitectura de Redes Neuronales: El Perceptrón Multicapa

En esta práctica vamos a ver cómo funciona realmente el Perceptrón Multicapa (MLP) y cómo logra aprender fronteras de decisión que no son simples líneas rectas. A diferencia del perceptrón simple, el MLP resuelve problemas mucho más complejos gracias a las capas ocultas. Durante las tareas, hemos probado distintos ajustes (como el número de neuronas y la función de activación) para ver cómo afectan a la rapidez con la que aprende el modelo y a lo bien que generaliza.

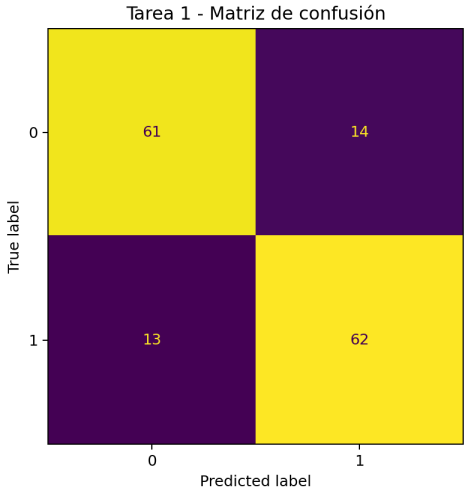
1.1. Tarea 1: La barrera de la linealidad — El Perceptrón Simple

En esta primera tarea, hemos probado un Perceptrón simple (un modelo puro, sin capas ocultas, usando la arquitectura `hidden_layer_sizes=()`) contra el dataset sintético `make_moons`. Al entrenarlo saca en test alrededor de un 82 %. Puede parecer un buen número de primeras, pero basta con mirar las gráficas para darse cuenta de que el modelo falla sistemáticamente en las mismas zonas.

1.1.1. Resultados y Análisis Estructural



(a) Frontera de decisión lineal.



(b) Matriz de confusión.

Figura 1.1: Evaluación del rendimiento en un perceptrón de una sola capa.

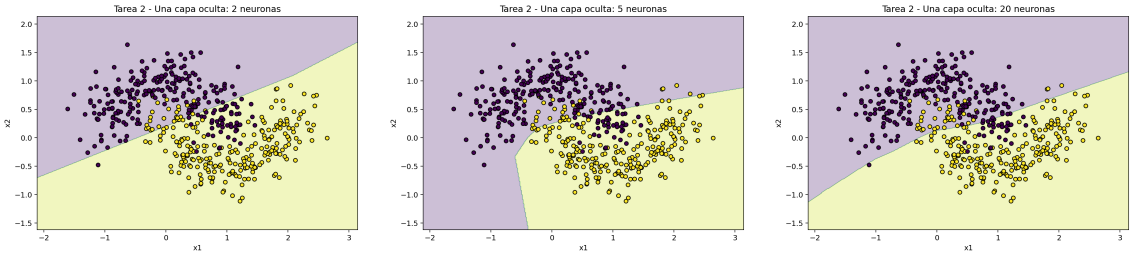
Viendo la frontera de decisión (Figura 1.1a), el motivo de esto está muy claro. Nuestros datos tienen forma de dos medias lunas entrelazadas (como un yin-yang), pero lo que ha aprendido el modelo es a trazar una recta que corta las figuras de forma totalmente plana. Matemáticamente es imposible que una sola línea recta separe perfectamente estas dos clases, y de ahí salen los errores de la matriz de confusión (Figura 1.1b).

El fallo aquí no es que el modelo necesite entrenar más tiempo, es una limitación de diseño en la propia red. Como no tiene capas ocultas, el Perceptrón simple solo saca combinaciones lineales de los datos. Resumiendo: solo puede dibujar rectas, hiperplanos, y figuras planas.

Básicamente, tenemos un problema de *underfitting* de manual por ser un modelo demasiado simple. Si queremos separar distribuciones que no se pueden cruzar con tiralíneas, el modelo necesita aprender patrones no lineales, y para eso hacen falta capas ocultas.

1.2. Tarea 2: Impacto de la capacidad en la capa oculta

Visto que el modelo lineal se queda corto, el siguiente paso ha sido añadir una capa oculta al MLP para que pueda manejar curvas. Hemos probado tres tamaños distintos (2, 5 y 20 neuronas) para ver visualmente cómo va cambiando la frontera a medida que le damos más músculo analítico.



(a) 2 neuronas: Underfitting. (b) 5 neuronas: Transición. (c) 20 neuronas: Ajuste óptimo.

Figura 1.2: Evolución de la frontera de decisión según la densidad de la red.

Tabla 1.1: Resultados experimentales según la arquitectura.

Arquitectura	Acc. Test	Iteraciones	Observación
MLP(2)	0.8200	648	Capacidad insuficiente
MLP(5)	0.8400	726	Mayor esfuerzo de optimización
MLP(20)	0.8533	406	Convergencia eficiente

Configuración con 2 neuronas

Con solo dos neuronas el modelo es algo más flexible que la recta pura de antes, pero sigue quedándose a medias. Si miramos la gráfica, la frontera solo forma una especie de "V". No tiene soltura suficiente para rodear las lunas enteras, así que seguimos cayendo en *underfitting*.

Configuración con 5 neuronas

Saltando a 5 neuronas, la cosa cambia mucho y ya casi capta la forma del problema. No obstante, se nota a simple vista que la frontera no es curva lisa. Como

estamos usando la activación ReLU, lo que hace el modelo realmente es juntar varios trozos rectos (los 5 «codos» de las neuronas) para intentar envolver las lunas. Arregla el problema inicial, pero visualmente queda con un polígono a trompicones típico de la ReLU.

Configuración con 20 neuronas

Con 20 neuronas le sobra capacidad para dibujar la frontera usando todos esos tramos lineales pequeñitos que se funden. Lo curioso es que, en un problema 2D tan simple, meter 20 neuronas no ha provocado que el modelo memorice como loco y la frontera empiece a hacer "islas" raras atrapando ruido (*overfitting*). Sigue generalizando bien, de hecho la precisión en train (86.00 %) y test (85.33 %) son casi la misma, aunque aquí la ganancia respecto a la anterior empieza a ser residual.

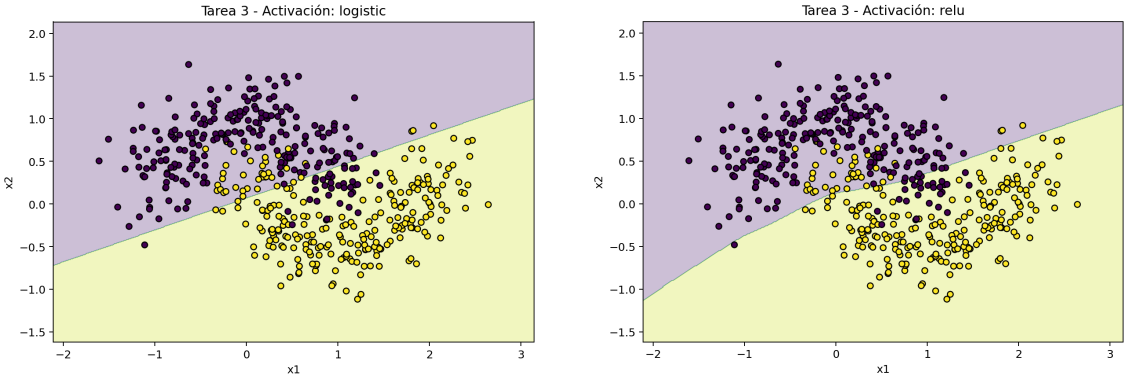
Conclusión

Queda comprobado que meter capas ocultas le quita el cepo lineal a la red, adaptándola a la forma real del problema (en nuestro caso usando cachos rectos que es como opera ReLU). La clave de todo es el equilibrio: no pasarnos de simples como para arrastrar *underfitting*, pero tampoco inflar el número de neuronas cuando una red ligera lo saca casi igual y no consume tanto.

1.3. Tarea 3: Dinámica de Aprendizaje — ReLU vs. Sigmoide

Manteniendo fija la capa de 20 neuronas, hemos puesto cara a cara dos funciones de activación míticas: la típica curva logística (*Sigmoide*) y la clásica lineal de hoy en día (*ReLU*). Queríamos ver cuánto cambia el contorno dibujado y la velocidad de entrenamiento solo con cambiar la activación bajo la misma red.

1.3.1. Fronteras de decisión



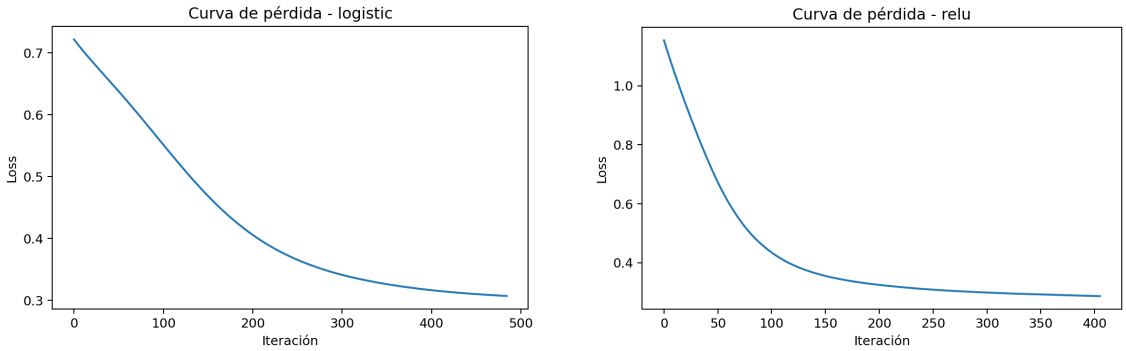
(a) Frontera con Sigmoide.

(b) Frontera con ReLU.

Figura 1.3: Comparación visual de la frontera de decisión según la activación.

Las dos activaciones separan las lunas más o menos igual de bien, pero haciendo trampas de diferente manera. La Sigmoide saca fronteras mucho más suaves y bonitas, que fluyen más, gracias a sus transiciones entre 0 y 1. ReLU, como ya vimos, tira de lógica de trozos rectos y se vuelve mucho más angulosa, pero funcionalmente cumple igual.

1.3.2. Curvas de pérdida y velocidad de convergencia



(a) Curva de pérdida con Sigmoide.

(b) Curva de pérdida con ReLU.

Figura 1.4: Comparativa de la velocidad de aprendizaje.

Tabla 1.2: Métricas comparativas por función de activación (arquitectura `(20,)`).

Activación	Acc. Train	Acc. Test	Iteraciones	Loss final
Sigmoide (logistic)	0.8657	0.8400	485	0.3071
ReLU	0.8600	0.8533	406	0.2870

Análisis

Viendo los números (Tabla 1.2) comprobamos lo que ya se dejaba ver en las curvas (Figura 1.4): ReLU vuela y llega al tope en **406 iteraciones**, ganándole un 16 % limpio a las **485** que se chupa la Sigmoide.

El motivo por el que la Sigmoide da más pereza computacional se llama *vanishing gradient*. Al aplastar siempre su salida entre $(0, 1)$, si entran valores muy extremos la curva se plana muchísimo, casi a cero. Eso hunde el *backpropagation* (al multiplicar valores de cero) y frena el aprendizaje. ReLU por su parte tiene siempre la derivada plantada en 1 para lo positivo; el gradiente no se estanca durante las pasadas y el ajuste corre libremente.

Al fin y al cabo, a nivel generalización ambas sacan sobre un ~84-85 % en prueba porque lunas 2D es algo asequible. Pero está claro que en cuanto se meta esto en deep-learning fuerte de más capas, el atasco en la Sigmoide mataría el sistema, siendo ese el motivo por el cual ReLU arrasó y es hoy la respuesta fácil con keras o sklearn en arquitecturas reales.

1.4. Tarea 4: Reconocimiento de Dígitos y Parsimonia

El último mini-reto era unificar conceptos probando a clasificar el dataset de imágenes `load_digits`. Teníamos que superar sí o sí el 95 % en test tirando por la navaja de Ockham (el modelo que rinda más fácil sin comer recursos). Hemos pasado una barrida de seis configuraciones de capas.

Tabla 1.3: Métricas obtenidas para distintas arquitecturas en el dataset de dígitos.

Arquitectura	Acc. Train	Acc. Test	Iteraciones	Profundidad	Meta
MLP(20)	1.0000	0.9644	276	1 capa	✓
MLP(50)	1.0000	0.9711	180	1 capa	✓
MLP(100)	1.0000	0.9800	147	1 capa	✓
MLP(30, 15)	1.0000	0.9600	161	2 capas	✓
MLP(50, 20)	1.0000	0.9756	131	2 capas	✓
MLP(64, 32)	1.0000	0.9667	110	2 capas	✓

1.4.1. Análisis de sobreajuste

Lo más llamativo al ver la Tabla 1.3 es que ni una sola métrica perdona un mísero 1 %: literal **todas las variantes alcanzan el 100 % sobre los datos de validación de entrenamiento** (`acc_train = 1.0000`). Se saben todas las fotos de memoria, haciendo puramente *overfitting*. Nos salva que las predicciones con los datos sin catalogar caen un ridículo margen de error al rededor del 3 % y retienen todo lo bueno generalizando.

Es algo esperable en un array de solo 1.350 filas y redes que tienen miles de nodos para tragárselo. Por suerte lo atenuamos de entrada dejando el parámetro de ‘sklearn‘ `alpha=1e-4` apretando la norma L2 en los pesos más inflados del grafo.

1.4.2. Elección del modelo: Principio de Parsimonia

Sabemos que si le tiramos la salvajada de $MLP(100)$ te levanta un 98 %, pero hay que guardar coherencia: elegimos a pies juntillas la variante **MLP(20)**. Simple y llánamente fue la primera de las pequeñas en llegar a la cuota del 95 % empleando ~1.500 parámetros de nada (contra los más de 7.500 de la más grande). Nos viene que ni pintado para mantener control extra de escalabilidad frente al futuro a menos potencia computacional requerida.

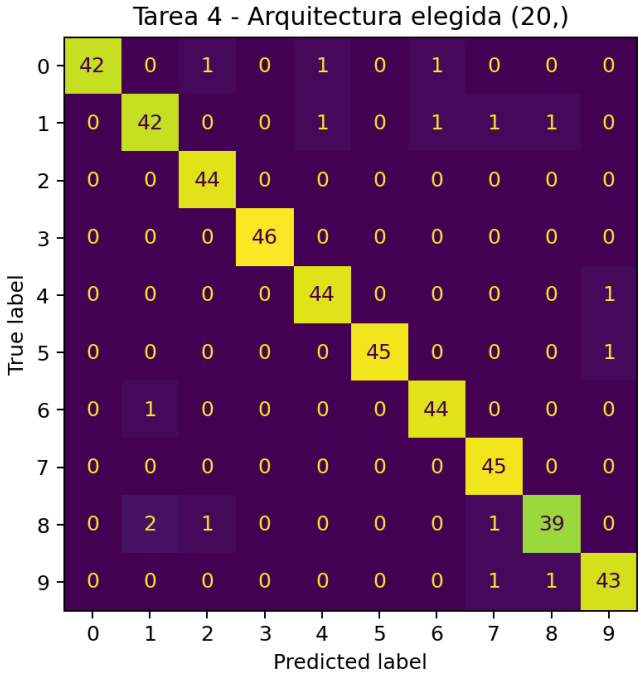


Figura 1.5: Matriz de confusión de la arquitectura **MLP(20)**. El desempeño es casi impecable en todas las clases; los escasos errores se concentran en pares de dígitos visualmente similares.

Propuesta Ejecutiva de Despliegue

A la atención de: Dirección Operativa / Product Owner
De: Equipo de Data Science e Ingeniería ML
Asunto: Recomendación para el pase a producción del clasificador de dígitos

A la vista del tracking final de los experimentos, traemos los resultados para decidir qué motor pasar a entorno productivo con tal de agilizar nuestro pipeline actual:

1. Modelo Seleccionado

Nuestra firma va para poner en despliegue vivo el modelo **MLP(20)** (que es un esquema plano de 20 neuronas en oculta y relus). Se asienta establemente en **96.44 %** de fiabilidad validada frente a sets que no había atacado nunca, batiendo holgadamente la cota para darnos la luz verde del 95 %.

2. Justificación Técnica: Principio de Parsimonia

Hemos apostado por rascar peso en red antes que arañar algún cero más de exactitud que no rentaba. Es cierto que forzando el servidor a una red de cien en bloque $MLP(100)$ habríamos trepado a un 98 %, pero nuestra variante gasta casi **cinco veces menos tamaño final en binario y params**. A final de mes en el hardware eso nos regala:

- **Tiempos de latencia absurdamente bajos:** Para clavar las respuestas asíncronas HTTP/REST de inmediato.
- **Muy poca RAM consumida:** Abriendo opciones sin refactors gordos a empaquetarlo localmente en endpoints móviles / IoT a posteriori gracias al margen.
- **Mejor mantenibilidad sin desastres por Data-Drift:** A menos lio numérico y más simple, sabemos que se va a desviar con menos estruendo cuando empiecen los ruidos y artefactos pesados fuera de laboratorio atacando las capturas diariarias de los clientes.

3. Consideraciones de Despliegue

Vale aclarar que esto es en cristalera; las pruebas arrojan tasas perfectas de match sobre sets vírgenes por sobreaprender internamente (train acc 100 %), y aunque nuestra rutina L2 le corta las alas al overfit y salva los trastos generalizando, recomendamos el típico **shadow deploy para curarnos en salud** al salir al ecosistema ruidoso real durante los tres primeros días de margen que arranquemos servicio en sombra, lo vemos contra los targets reales offline, y, si nadie suelta las alarmas redirigimos vivo.